```
Output:
Fatal error: Cannot use positional argument after argument unpacking
```

However, named parameters can be used after unpacking (PHP 8.1 or later):

```
$albumAndArtist = ['Left Hand Path', 'Entombed'];

knowYourClassicMetal(...$albumAndArtist, year: '1990', label: 'Combat
Records');

Output:
Left Hand Path by Entombed, released in 1990 on Combat Records
```

Care also needs to be taken over possible collisions between named and positional arguments. Remember earlier when we looked at how PHP will not allow duplicate named parameters in a call? When mixing named with positional or unpacked it is possible for this to happen in more subtle ways, which could result in code that only works sometimes – the hardest of all to debug!

The following code is legal syntax, it will not trigger any errors when linted, but the first argument corresponds to the `$album` parameter in the function declaration and the explicitly named `album` argument on the end triggers an error:

```
knowYourClassicMetal('Somewhere in Time', artist: 'Iron Maiden', year:
'1985', label: 'EMI', album: 'Somewhere in Time');

Output:
Fatal error: Uncaught Error: Named parameter $album overwrites previous
argument
```

Things can get even more obscure when utilizing an associative array with string keys as parameter names. Remember the rule about not putting positional arguments after named? That also applies to unpacked arrays. Observe:

```
knowYourClassicMetal(...['album' => 'Master of Puppets','artist' =>
'Metallica',
   'year' => '1986', 'label'=> 'Elektra Records']);
knowYourClassicMetal(...['Master of Puppets', 'artist' => 'Metallica',
   'year' => '1986', 'label'=> 'Elektra Records']);
```